Ira A. Fulton Schools of Engineering
Arizona State University

CSE 568: Biocomputing

Introduction to Graph Coloring Project

Purpose

The purpose of this project is to build foundational experience with evolutionary computation by applying a genetic algorithm to a classic combinatorial optimization problem. Graph coloring provides a practical setting for examining how problem representation, fitness evaluation, evolutionary operators, and parameter choices influence the quality of solutions. Developing this model strengthens your ability to translate computational constraints into an evolutionary search process, an approach relevant to scheduling, resource allocation, network design, and other optimization problems.

Objectives

Learners will be able to:

- **2.2**: Apply evolutionary computation algorithms to solve optimization problems.
- **2.3**: Implement and evaluate a genetic algorithm to solve graph-coloring problems.

Technology Requirements

- A local Python development environment
- Python 3
- Required Python package:
  - Deap
- Optional local testing package:
  - pytest
- Access to Gradescope

Description

In this project, you will implement a genetic algorithm in Python to solve a graph-coloring problem using the DEAP evolutionary computation library. You will represent graphs with adjacency lists, read graph data from files, and encode candidate colorings as genomes. You will then create a custom fitness function, evolve graph colorings, and tune genetic algorithm parameters to improve solution quality.

The project has three main goals:

- Parse graph data into adjacency lists and encode colorings as genomes.
- Implement a custom fitness function to evaluate graph colorings.
- Use DEAP and tune genetic algorithm parameters to evolve high-quality solutions.

Directions

Before getting started, refer to the additional resources and files provided in the course to complete your project accurately:

- Project Overview Video
- Project ZIP File

Required Functions

Your file must implement:

```
def init_graph(file_path)
def decode_individual(graph, individual)
def eval_graph(graph, individual)
def run_ga(file_path, population_size=300, ngen=200, cxpb=0.8,
mutpb=0.3)
```

Graph File Format

Graph files follow this format:

```
<number_of_nodes>
<number_of_colors>
u v
u v
...
```

```
Example:
4
3
1 3
0 2
2 1
```

Question 1 — Graph Parsing

- Implement init_graph(file_path). Your function should read the graph file, construct the Graph object, add all edges, and return the completed graph.

Question 2 — Fitness Function

- Implement `decode_individual(graph, individual)` and `eval_graph(graph, individual)`.
- Individuals are binary genomes encoding graph colors.
  You must decode the genome, reject invalid color encodings, count satisfied edges, and return fitness as a tuple.

Question 3 — Genetic Algorithm

- Implement `run_ga(...)`. Use DEAP to evolve graph colorings using crossover, mutation, and selection.

Local Testing

- You may test locally using:
  pytest tests/ -v

Gradescope Grading

- Your project will be graded automatically using public and hidden tests.

Common Mistakes

- Returning scalar fitness instead of a tuple
- Invalid color handling
- Double-counting edges
- Wrong submission filename

Submission Directions for Deliverables

You must submit your Introduction to Graph Coloring Project deliverable through **Gradescope**. Carefully review submission directions outlined in this overview document in order to correctly earn credit for your work. Learners may not email or use other means to submit any assignment or project for review, including feedback, and grading.

The Introduction to Graph Coloring Project includes one (1) deliverable:

- **student_solution.py:** Submit one (1) Python solution. Do not submit notebooks, ZIP files, PDFs, or multiple files.

Gradescope Submission

Your submission will be reviewed by the course team, and then, after the due date has passed, your score will be populated from Gradescope into your grade.

1. Go to the Canvas Assignment, "**Submission: Introduction to Graph Coloring Project**".
2. Click the "**Load Submission…in new window**" button.
3. Once in Gradescope, select the project titled "**Introduction to Graph Coloring Project,**" and a pop-up window will appear.
4. In the pop-up window, submit your "**student_solution.py**" file for grading.
5. If needed: to resubmit the project in Gradescope:
    a. Return to the Canvas submission and open Gradescope.
    b. You will be navigated to the "Autograder Results" page (if it is not your first submission).
    c. Click the "**Resubmit**" button on the bottom right corner of the page and repeat the process from Step 3.

Evaluation

Please review the table for how your project will be graded. The project deliverable will be evaluated by the autograder based on each criterion and will receive a total score.

Deliverables missing any part of the project will be graded based on what was submitted against the grading criteria. Missing parts submitted after the deadline will not be graded.

Review the course syllabus for details regarding late penalties.

Gradescope Weights

The autograder awards 100 total points.

Questions	Points
Q1: init_graph sample graph parsing	15
Q1: init_graph additional graph parsing	15
Q2: decode_individual valid encodings	15
Q2: eval_graph satisfied-edge fitness	20
Q2: invalid color encoding returns zero	10
Q3: run_ga sample graph behavior	15
Q3: run_ga hidden graph behavior	10
Total	**100**